TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
CÔNG NGHỆ PHẦN MỀM

BrewLite

Ứng dụng đặt cà phê không dùng tiền mặt
Triển khai 10 task theo Agile Scrum

BẢN DEMO – REVIEW LẦN 1

Thành viên	MSSV	Phân công
Bảo Tín	Bổ sung	Task 1, 6
Phú	Bổ sung	Task 2, 9
Bảo Thiện	Bổ sung	Task 3, 5
Lợi	Bổ sung	Task 4, 7
Thuận	Bổ sung	Task 8, 10

GVHD: Từ Lãng Phiêu (theo mẫu, nhóm xác nhận)
Học kỳ I – Năm học 2026–2027
TP. Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Tóm tắt và phạm vi báo cáo **3**
1.1 Mốc đối chiếu . . . 3

2 Sản phẩm, yêu cầu và kiến trúc **3**
2.1 Người dùng và phạm vi MVP . . . 3
2.2 Yêu cầu chức năng và phi chức năng . . . 3
2.3 Kiến trúc triển khai . . . 4
2.4 Kịch bản nghiệp vụ . . . 4

3 Quy trình Agile Scrum **4**

4 Phân công và đối chiếu Git **5**

5 Tiêu chí và minh chứng cho 10 task **6**
5.1 Task 1 — Khởi tạo dự án và cấu trúc . . . 6
5.2 Task 2 — API danh sách sản phẩm . . . 6
5.3 Task 3 — Trang Menu (Frontend) . . . 6
5.4 Task 4 — Chi tiết và tùy chọn sản phẩm . . . 7
5.5 Task 5 — Giỏ hàng (Cart) . . . 7
5.6 Task 6 — API tạo đơn hàng . . . 7
5.7 Task 7 — Đăng ký / Đăng nhập (JWT) . . . 8
5.8 Task 8 — Thanh toán không tiền mặt . . . 8
5.9 Task 9 — Xác nhận, lịch sử đơn và bàn giao . . . 8
5.10 Task 10 — Nghiệp vụ backend (đề §9) . . . 9

6 Task 10: giải thích và điều kiện nghiệm thu **10**
6.1 Máy trạng thái đơn hàng . . . 10
6.2 Idempotency, giao dịch và tồn kho . . . 10
6.3 Khuyến mãi và điểm thưởng . . . 10

7 Kiểm thử, Definition of Done và bàn giao **11**
7.1 Kết quả lịch sử và giới hạn . . . 11
7.2 Ma trận nghiệm thu cuối cùng . . . 11
7.3 Kịch bản demo bàn giao . . . 11
7.4 Retrospective và cải tiến đề xuất . . . 12

8 Các mục cần nhóm chốt sau review 1 **12**

9 Nguồn và cách tái kiểm tra **12**

1 Tóm tắt và phạm vi báo cáo

BrewLite phục vụ khách hàng chọn đồ uống, tùy chọn size và topping, quản lý giỏ hàng, đăng nhập, tạo đơn, thanh toán giả lập bằng Ví/Thẻ và theo dõi lịch sử đơn. Frontend dùng Next.js; backend dùng NestJS; dữ liệu lưu bằng PostgreSQL qua Prisma. Docker Compose hỗ trợ bàn giao ba dịch vụ frontend, backend và cơ sở dữ liệu.

Bản báo cáo bám đề `docs/BrewLite-CongNghePhanMem-fixed.md`, đặc biệt mục 8, 9 và 10. Mười task có tổng trọng số 10 điểm, mỗi task 1 điểm; đây là trọng số đề bài, không phải điểm tự nghiệm thu. Bố cục bìa, logo SGU, đầu trang và chân trang được điều chỉnh từ mẫu LaTeX đã cung cấp. Các đoạn MATLAB và nội dung thống kê không liên quan trong mẫu không được dùng.

Bản review này phân biệt ba loại thông tin: yêu cầu của đề, minh chứng trong repository, và thông tin còn cần nhóm xác nhận. Việc có code hoặc commit không tự chứng minh toàn bộ Definition of Done. Tên thành viên và phân công do nhóm cung cấp; MSSV chưa có nên để trống, không suy diễn từ tài khoản Git.

1.1 Mốc đối chiếu

Ngày kiểm tra: 23/10/2026. Nhánh: `develop-agile`. HEAD: `f6caead98e05dcf67ec29487 26f3a4324f1c480d`. Các remote-tracking ref được đọc tại máy; phiên này không fetch, không khẳng định đã đồng bộ máy chủ.

Mẫu nguồn đang là thư mục chưa được Git theo dõi; bản demo chỉ tạo file trong thư mục riêng. Các kết quả test được dẫn ở phần sau là kết quả lịch sử từ tài liệu/log, không phải test ứng dụng được chạy lại trong phiên soạn báo cáo.

2 Sản phẩm, yêu cầu và kiến trúc

2.1 Người dùng và phạm vi MVP

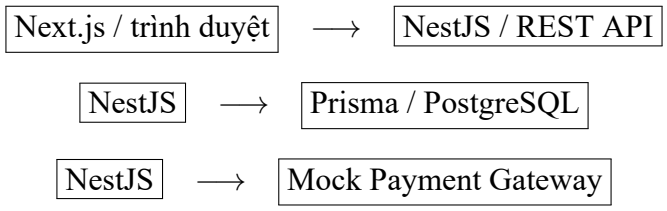
Khách hàng xem menu, đăng ký/đăng nhập, đặt đồ uống và kiểm tra đơn của mình. Barista và Admin thao tác nghiệp vụ quầy qua API trong phạm vi minh chứng Task 10; bản web tại nhánh này không có màn hình quầy/Admin. Product Owner xác định ưu tiên và nghiệm thu; Scrum Master hỗ trợ quy trình; giảng viên đánh giá sản phẩm và hồ sơ.

2.2 Yêu cầu chức năng và phi chức năng

1. Menu có ảnh, tên, giá; giao diện xử lý loading, empty và lỗi.
2. Chi tiết món cho chọn size S/M/L và topping, hiển thị giá thay đổi.
3. Giỏ hỗ trợ thêm, sửa số lượng, xóa và tính tổng; có badge số lượng.
4. Tài khoản dùng bcrypt và JWT; route đặt đơn được bảo vệ.
5. Tạo đơn PENDING trước khi thanh toán; thành công sang PAID, lỗi sang PAYMENT_FAILED.
6. Xác nhận có mã đơn và trạng thái; lịch sử chỉ trả đơn của khách hiện tại.

Yêu cầu phi chức năng: API phản hồi dưới 500 ms với dữ liệu mẫu; validate đầu vào, băm mật khẩu, bảo vệ bằng JWT; sử dụng được trên trình duyệt. Cần bổ sung ma trận trình duyệt, điều kiện đo và ảnh minh chứng; một lần đo với dữ liệu mẫu không đại diện kiểm thử tải.

2.3 Kiến trúc triển khai



Frontend quản lý tương tác và giỏ; backend xác thực, tính lại giá, kiểm soát trạng thái, tồn kho và thanh toán. Không tin giá tiền do client gửi. Cổng thanh toán là mô phỏng, không đại diện giao dịch với nhà cung cấp thực.

2.4 Kịch bản nghiệp vụ

Khách mở menu, chọn món và tùy chọn, thêm giỏ, điều chỉnh số lượng, đăng nhập rồi thanh toán. Frontend gọi tạo đơn trước, nhận mã đơn PENDING, sau đó gọi thanh toán với Idempotency-Key. Thành công thì hiển thị biên lai và lịch sử; thất bại thì giữ giỏ, cho phép thử lại trên đơn cũ. Nhân viên cập nhật tiến trình pha chế bằng API đúng quyền.

3 Quy trình Agile Scrum

Sprint	Mục tiêu theo đề	Task
1	Nền tảng và hiển thị menu	1, 2, 3, 4
2	Giỏ hàng, đặt đơn, đăng nhập	5, 6, 7
3	Thanh toán, bàn giao, nghiệp vụ nâng cao	8, 9, 10

Đây là kế hoạch gợi ý của đề, không phải khẳng định lịch Sprint đã diễn ra. Product Backlog được lưu trong `docs/product-backlog.md`, hồ sơ nháp Scrum ở `docs/scrum-process.md`. Task hoàn thành cần đáp ứng tiêu chí và DoD, không chỉ được merge.

[Cần bổ sung: PO, Scrum Master, ngày bắt đầu/kết thúc Sprint, Sprint Planning, Daily Scrum, biên bản Review, người review và phản hồi thực tế.]

Burndown nên dùng số việc còn lại theo DoD mỗi ngày. Không lấy số commit làm phần trăm hoàn thành, không coi ngày merge là ngày nghiệm thu. Tài liệu Scrum cũ chứa hash trước khi viết lại lịch sử và đồ thị theo thời điểm tích hợp; cần nhóm xác nhận trước khi dùng làm burndown chính thức.

4 Phân công và đối chiếu Git

Phân công do nhóm cung cấp: Bảo Tín (Task 1, 6), Phú (Task 2, 9), Bảo Thiện (Task 3, 5), Lợi (Task 4, 7), Thuận (Task 8, 10). Các tài khoản Git của Thuận gồm `Thuan-Test`, `Trần Vĩnh Thuận`, `Thuan-Lap` và `Thuan-AFK666`, được gộp thành một thành viên. Mỗi dòng dưới đây nêu commit do chính người phụ trách tạo; nhánh chứa commit chỉ là bằng chứng truy cập, không xác định nơi viết code.

Phân công	Commit / tác giả	Nhánh	Nội dung
Bảo Tín, Task 1	`44e3784` Saturlen	main (upload 13/10)	README, Compose, Dockerfile, mã backend ban đầu.
Phú, Task 2	`a358930` phu0111	a1task2-9	Đặc tả API sản phẩm; `5d92e1f` hoàn thiện.
Bảo Thiện, Task 3	`9ea0933` baothien-s	thien35; merge `862bf65`	Menu Next.js, gọi API, trạng thái tải/rỗng.
Lợi, Task 4	`32831dc` DuVinhLoi	feat/task4-task7; merge `eaf33f8`	Chi tiết món, loading/lỗi, test.
Bảo Thiện, Task 5	`9ea0933` baothien-s	thien35; merge `862bf65`	Giỏ hàng, tổng tiền, badge.
Bảo Tín, Task 6	`44e3784` Saturlen	main (upload 13/10)	Mã tạo đơn: controller, DTO, tính giá.
Lợi, Task 7	`32831dc` DuVinhLoi	feat/task4-task7; merge `eaf33f8`	JWT, chuẩn hóa email, test xác thực.
Thuận, Task 8	`bd1adc3` Thuan-Test	feature/task-8-10-payment-order; merge `03cb553`	Thanh toán mock, đơn PENDING → PAID/PAYMENT_FAILED.
Phú, Task 9	`ed2e4ee` phu0111	a1task2-9	Tài liệu xác nhận đơn và lịch sử đơn.
Thuận, Task 10	`bd1adc3` Thuan-Test	feature/task-8-10-payment-order; merge `03cb553`	State Machine, idempotency, tồn kho, loyalty.

5 Tiêu chí và minh chứng cho 10 task

Các tiểu mục sau chuyển biên nội dung minh chứng từ `docs/minh-chung-10-task.md` và đối chiếu tiêu chí ở đề. Các test và ảnh được liệt kê là vị trí kiểm tra/kế hoạch chứng minh; không mặc định tất cả ảnh đã tồn tại hoặc test đã chạy ở HEAD hiện tại. Bảng Git phía trên thay thế các thống kê tác giả theo snapshot cũ trong tài liệu nguồn.

5.1 Task 1 — Khởi tạo dự án và cấu trúc

Đề §8: monorepo `frontend` (Next.js) + `backend` (NestJS), chạy được ”Hello”, có README, Git, `.env.example`.

• Postman `Task01`: `T01.1 GET{{webUrl}}/` trả 200 HTML có chữ ”BrewLite”; `T01.2 GET{{baseUrl}}/products` trả 200 JSON.

• File: `README.md`, `.env.example`, `frontend/package.json`, `backend/package.json`, `docker-compose.yml`.

• Test: `backend/test/bootstrap-env.spec.ts` (thiếu `DATABASE_URL` thì dừng), `backend/test/task47-auth.spec.ts` (thiếu `JWT_SECRET` thì dừng).

• Ảnh cần chụp: cây thư mục repo; `gitlog--oneline`; Runner Postman thư mục `Task01`.

5.2 Task 2 — API danh sách sản phẩm

Đề §8: `GET/products` trả JSON menu (id, tên, giá, ảnh).

• Postman `Task02`: `T02.1` trả mảng, mỗi món có `id`, `name`, `price`, `imageUrl`, không có món `stock=0`; `T02.2` `GET/products/:id` trả đúng món; `T02.3` id không có trả 404.

• File: `backend/src/product/product.controller.ts`, `backend/prisma/seed.ts`.

• Test: `backend/test/order-product.spec.ts` (ca ”product list excludes stock-0”, ”product detail unknown → NotFound”).

• Ảnh cần chụp: Postman `T02.1` (tab Body và Test Results).

5.3 Task 3 — Trang Menu (Frontend)

Đề §8: Next.js gọi `/products`, render lưới sản phẩm, có loading / empty state.

• Postman `Task03`: `T03.1` trang `/` trả 200; `T03.2` ảnh `imageUrl` của món tải được (`image/svg+xml`).

• File: `frontend/src/components/ProductGrid.tsx`, `frontend/src/lib/api.ts` (`getProducts`).

• Test: `frontend/src/components/ProductGrid.test.ts` (loading, empty, lỗi + ”Thử lại”), `frontend/src/lib/api.test.ts`.

• Ảnh cần chụp: trang Menu `http://localhost:3001` có lưới món; trạng thái loading (DevTools → Network → Slow 3G); trạng thái lỗi (tắt backend: `dockercomposestopbackend`).

5.4 Task 4 — Chi tiết và tùy chọn sản phẩm

Đề §8: chọn size (S/M/L) và topping, tính giá theo tùy chọn.

• Postman `Task04`: `T04.1` size L + Trân châu + Kem cheese thì `unitPrice` = giá gốc + 10.000 + 13.000; `T04.2` size S không topping thì bằng giá gốc; `T04.3` topping lạ trả 400 `Unknowntopping`. Backend tính giá bằng cùng bảng giá với frontend.

• File: `frontend/src/components/ProductDetailModal.tsx`, `frontend/src/lib/pricing.ts`, `backend/src/order/pricing.ts`.

• Test: `frontend/src/lib/pricing.test.ts`, `frontend/src/components/ProductDetailModal.test.ts`, `frontend/src/lib/product-detail.test.ts`, `backend/test/pricing-table.spec.ts` (bảng giá hai phía trùng nhau).

• Ảnh cần chụp: modal chi tiết món trước và sau khi đổi size/topping (giá thay đổi).

5.5 Task 5 — Giỏ hàng (Cart)

Đề §8: thêm / sửa số lượng / xóa, lưu state (Zustand), tự tính tổng, badge số lượng.

• Postman `Task05` (phía API của giỏ): `T05.1` giỏ 2 dòng thì `total` = tổng `quantity×unitPrice`; `T05.2` 21 dòng trả 400; `T05.3` 100 ly một dòng trả 400 (giới hạn 20 dòng, 1–99 ly trùng giới hạn của giỏ).

• File: `frontend/src/store/cart.ts`, `frontend/src/components/cart/CartView.tsx`, `frontend/src/components/HeaderView.tsx`.

• Test: `frontend/src/store/cart.test.ts`, `frontend/src/lib/cart-limits.test.ts`, `frontend/src/components/cart/CartView.test.ts`, `frontend/src/components/QtyStepper.test.ts`, `frontend/src/components/HeaderView.test.ts`.

• Ảnh cần chụp: giỏ hàng có 2 món và tổng tiền; badge trên header; tải lại trang (F5) giỏ vẫn còn.

5.6 Task 6 — API tạo đơn hàng

Đề §8: `POST/orders` nhận giỏ hàng, validate (class-validator), lưu `PENDING`, trả mã đơn.

• Postman `Task06`: `T06.1` không token trả 401; `T06.2` giỏ rỗng trả 400; `T06.3` size `XL` trả 400; `T06.4` món không có trả 404; `T06.5` đơn hợp lệ trả 201, `status` `PENDING`, có `id`, gửi kèm `unitPrice:1` vẫn bị server bỏ qua và tự tính giá; `T06.6` token quầy trả 403.

• File: `backend/src/order/order.controller.ts`, `backend/src/order/dto/create-order.dto.ts`, `backend/src/order/pricing.ts`.

• Test: `backend/test/order-product.spec.ts`, `backend/test/pricing-table.spec.ts`, `backend/test/customer-only.spec.ts`.

• Ảnh cần chụp: Postman `T06.5` (Body có `id`, `status:"PENDING"`) và `T06.2` (400 với thông báo lỗi validate).

5.7 Task 7 — Đăng ký / Đăng nhập (JWT)

Đề §8: `POST/auth/register`, `/auth/login`; băm mật khẩu (bcrypt); phát JWT; guard bảo vệ route đặt đơn.

• Postman `Task07`: `T07.1` đăng ký trả 201, không có `passwordHash`; `T07.2` trùng email (viết hoa) trả 409; `T07.3` email sai, mật khẩu ngắn trả 400; `T07.4` đăng nhập trả JWT 3 phần, `role` `CUSTOMER`; `T07.5` sai mật khẩu và `T07.6` email không tồn tại cùng trả 401 `Invalidcredentials`; `T07.7` token giả trả 401. Guard route đặt đơn: `T06.1`.

• File: `backend/src/auth/auth.controller.ts`, `backend/src/auth/auth.service.ts` (bcrypt), `backend/src/auth/jwt-auth.guard.ts`, `frontend/src/app/login/page.tsx`.

• Test: `backend/test/auth.spec.ts`, `backend/test/task47-auth.spec.ts`, `backend/test/roles.spec.ts`, `frontend/src/lib/login.test.ts`, `frontend/src/lib/session.test.ts`, `frontend/src/components/LoginView.test.ts`.

• Ảnh cần chụp: Postman `T07.4` (có `accessToken`), `T07.5` (401); bảng `User` trong DB có `passwordHash` dạng `$2a$…` (`dockercomposeexecdbpsql-Ubrewlite-c'selectemail,left("passwordHash",7)from"User"limit3;'`).

5.8 Task 8 — Thanh toán không tiền mặt

Đề §8 và §4.3: chọn Ví/Thẻ, gọi POST/payments; đơn PENDING → PAID khi thành công, PAYMENT_FAILED khi lỗi, giỏ giữ nguyên.

• Postman Task08: T08.1 trả bằng Ví trả 201 PAID kèm paymentId; T08.2 đọc lại đơn thấy PAID; T08.3–T08.5 trả bằng Thẻ bị cổng giả từ chối trả 400 Paymentfailed, đơn sang PAYMENT_FAILED; T08.6 thử lại bằng Ví trả 201 PAID trên cùng đơn cũ; T08.7 thiếu Idempotency-Key trả 400; T08.8 phương thức CASH trả 400.

• File: backend/src/payment/payment.controller.ts, backend/src/payment/payment.service.ts, backend/src/payment/mock-payment-gateway.service.ts, frontend/src/components/PaymentForm.tsx.

• Test: backend/test/payment-failure.spec.ts, backend/test/order-payment.integration.spec.ts, frontend/src/components/PaymentForm.test.ts, frontend/src/lib/payment/api-gateway.test.ts, frontend/src/lib/payment/checkout-session.test.ts.

• Ảnh cần chụp: Postman T08.4 và T08.5; màn Thanh toán trên web báo lỗi khi chọn Thẻ, giỏ vẫn còn.

5.9 Task 9 — Xác nhận, lịch sử đơn và bàn giao

Đề §8: màn xác nhận (mã đơn, trạng thái), GET/orders/me, docker-compose chạy cả 3, README, demo end-to-end.

• Postman Task09: T09.1 GET/orders/:id có id, status, total, items; T09.2 GET/orders/me có các đơn vừa tạo, sắp xếp mới nhất trước, mỗi dòng có productName; T09.3, T09.4 khách B không thấy và không đọc được đơn của khách A (403); T09.5 mã đơn lạ trả 404; T09.6 trang /history của frontend trả 200.

• File: backend/src/order/order.controller.ts, frontend/src/app/history/page.tsx, frontend/src/components/PaymentSuccess.tsx, docker-compose.yml, README.md.

• Test: backend/test/order-product.spec.ts (ca "listMine returns newest-first"), backend/test/compose-env.spec.ts, frontend/src/components/HistoryView.test.ts, frontend/src/components/PaymentSuccess.test.ts, frontend/src/lib/history.test.ts, frontend/src/lib/use-order-poll.test.ts.

• Ảnh cần chụp: dockercomposeps có 3 service db, backend, frontend; màn biên lai có mã đơn và trạng thái PAID; trang /history.

5.10 Task 10 — Nghiệp vụ backend (đề §9)

Tiêu chí chấm của đề: test chứng minh (a) chặn chuyển trạng thái sai; (b) 2 lần cùng Idempotency-Key chỉ tạo 1 đơn; (c) đặt đồng thời không vượt tồn kho.

(a) Order State Machine. Postman Task10/(a):

• T10a.1 khách tự tiến đơn trả 403 Onlystaffcanadvanceorders.

• T10a.2 quầy nhảy cóc PAID → READY trả 400 InvalidtransitionPAID->READY.

• T10a.3, T10a.5, T10a.6 quầy tiến PREPARING → READY → COMPLETED, mỗi bước 200; T10a.4 khách hủy đơn đang PREPARING trả 400.

• T10a.7 COMPLETED → CANCELLED trả 400 InvalidtransitionCOMPLETED->CANCELLED; T10a.8 đơn vẫn COMPLETED.

• T10a.9 hàng đợi quầy GET/orders?status=PAID có đơn đã trả; T10a.10 quầy hủy đơn PAID trả 200 CANCELLED.

• Code: backend/src/state-machine/order-state.machine.ts (assertTransition), backend/src/payment/payment.service.ts (advanceOrder, cancelOrderAs).

• Test: backend/src/state-machine/order-state.machine.spec.ts, backend/test/order-advance.spec.ts, backend/test/barista-queue.spec.ts, frontend/src/lib/order-status.test.ts.

(b) Thanh toán idempotent. Postman Task10/(b):

• T10b.2, T10b.3 POST/orders 2 lần cùng key: lần 1 trả 201, lần 2 trả 200 cùng id, isCached: true.

• T10b.5, T10b.6 POST/payments 2 lần cùng key: lần 2 trả 200 cùng paymentId.

• T10b.7 điểm thưởng chỉ cộng 1 lần; T10b.8 tồn kho chỉ trừ 1 lần (đúng 2 ly); T10b.9 khách khác dùng lại key trả 422.

• Test: backend/test/order-payment.integration.spec.ts (ca "(b) same Idempotency-Key charges once").

(c) Tồn kho khi đặt đồng thời. Postman Task10/(c):

• T10c.1 ghi kho với version cũ trả 409 Stockchanged,reload (optimistic locking theo Product.version).

• T10c.2 trừ quá số còn lại trả 400 Stockcannotbenegative.

• T10c.3 khách gọi API kho trả 403.

• Postman Runner gửi request tuần tự nên không tạo được tải đồng thời. Phần "nhiều khách đặt cùng lúc" chứng minh bằng test backend/test/order-payment.integration.spec.ts (ca "(c) concurrent stock never negative") và backend/test/pg-payment.integration.spec.ts (Postgres thật, chạy khi có DATABASE_URL, cách chạy ở README.md).

(d) Khuyến mãi và điểm thưởng. Postman Task10/(d):

• T10d.3 trả với BREWLITE10: discount = floor(total×0.1), amount = total−discount, loyaltyPointsEarned = floor(amount×0.1/1000). Lần chạy 2026-10-21: đơn 58.000đ được giảm 5.800đ, trả 52.200đ, cộng 5 điểm.

• T10d.4 GET/users/me thấy điểm đã cộng; T10d.6 mã sai SAIMA thì discount = 0.

• Test: backend/test/order-payment.integration.spec.ts (voucher BREWLITE10).

Ảnh cần chụp cho Task 10: Runner Postman thư mục Task10 (đủ 4 thư mục con), cửa sổ terminal npxjestorder-stateorder-advanceorder-payment xanh; trang /history của khách tự chuyển bước sau khi chạy T10a.3–T10a.6.

6 Task 10: giải thích và điều kiện nghiệm thu

6.1 Máy trạng thái đơn hàng

$$\text{PENDING} \rightarrow \text{PAID} \rightarrow \text{PREPARING} \rightarrow \text{READY} \rightarrow \text{COMPLETED}$$

Nhánh lỗi: PENDING → PAYMENT_FAILED; thử lại: PAYMENT_FAILED → PENDING. Nhánh hủy: PENDING, PAID hoặc PAYMENT_FAILED → CANCELLED. COMPLETED và CANCELLED là trạng thái kết thúc. Mọi chuyển tiếp khác bị chặn; khách không được tự tiến trạng thái pha chế. Cần chứng minh cả kiểm tra quyền và kiểm tra chuyển trạng thái, vì hai điều kiện độc lập.

6.2 Idempotency, giao dịch và tồn kho

Cùng một yêu cầu với cùng khóa phải trả kết quả đã xử lý, không tạo đơn hoặc charge thêm. Cần kiểm tra mã đơn, mã thanh toán, tồn kho và điểm trước/sau, không chỉ HTTP 200. Cần phân biệt retry tạo đơn và retry thanh toán trên đơn đã có.

Optimistic locking dùng phiên bản bản ghi để phát hiện ghi cạnh tranh. Transaction đảm bảo nhóm thay đổi liên quan nhất quán. Minh chứng cần tạo yêu cầu đồng thời, xác nhận kho không âm và số giao dịch thành công phù hợp tồn kho. Runner tuần tự chỉ chứng minh phản hồi lỗi phiên bản cũ; không đủ chứng minh tải cạnh tranh. Test tích hợp PostgreSQL cần môi trường dữ liệu thật và bằng chứng chạy riêng, không dùng test bị skip để kết luận đạt.

6.3 Khuyến mãi và điểm thưởng

Theo quyết định nhóm trong đề đã hiệu chỉnh, mã BREWLITE10 giảm 10%:

$$discount = \lfloor total \times 0.1 \rfloor, \quad amount = total - discount$$

$$points = \lfloor amount \times 0.1/1000 \rfloor.$$

Ví dụ 58.000 đồng được giảm 5.800 đồng, thanh toán 52.200 đồng, cộng 5 điểm sau PAID. Đây là quy tắc nhóm chọn, không phải toàn bộ chi tiết bắt buộc của đề gốc. Cần kiểm tra mã sai, thanh toán lỗi và gửi lại cùng khóa không cộng điểm lặp.

7 Kiểm thử, Definition of Done và bàn giao

7.1 Kết quả lịch sử và giới hạn

Theo log `docs/postman/results/2026-10-21-run.txt` và tài liệu minh chứng ngày 21/10/2026: 74 request, 137 assertion, không thất bại; thời gian phản hồi lớn nhất 108 ms. Tài liệu ghi backend 170 test pass, 3 skip; frontend 733 test pass trong 46 file. Các con số unit test là trích dẫn tài liệu, chưa được chạy lại hoặc xác nhận bằng log độc lập trong phiên này. Không chuyển kết quả đó thành chứng nhận cho HEAD ngày 23/10.

7.2 Ma trận nghiệm thu cuối cùng

Điều kiện DoD	Cách xác nhận	Trạng thái review 1
Đủ tiêu chí 10 task	Đối chiếu từng tiểu mục với code, test và ảnh	Đã lập hồ sơ; cần nghiệm thu
Code chạy, không lỗi build	Chạy build frontend/backend và lưu log có SHA	Chưa chạy lại
Commit rõ, có review nhóm	Hash trong báo cáo; bổ sung PR/reviewer và kết luận	Git đã tra; review cần bổ sung
API validate, UI loading/lỗi	Các case âm và ảnh UI tương ứng	Đã chỉ vị trí minh chứng
README chạy được	Người khác làm theo README trên môi trường sạch	Cần ghi nhận lần nghiệm thu
Task 10 cạnh tranh	Test PostgreSQL thật, không skip, lưu kết quả	Cần log tại HEAD nghiệm thu

7.3 Kịch bản demo bàn giao

1. Dựng ba dịch vụ theo README; chụp trạng thái Compose và ghi phiên bản code.
2. Đăng ký/đăng nhập, xem menu, đổi size/topping, kiểm tra tổng giá.
3. Thêm hai món, sửa số lượng, xóa một món, kiểm tra badge và lưu giỏ.
4. Tạo đơn PENDING, thanh toán Ví, xem mã đơn PAID và lịch sử.
5. Demo cổng giả từ chối Thẻ; giữ giỏ và thử lại trên cùng đơn.
6. Chạy request sai quyền, sai trạng thái, cùng khóa; trình diễn tiến đơn qua API nhân viên.
7. Chạy test cạnh tranh với PostgreSQL, đối chiếu kho và điểm thưởng.
8. Bàn giao source, cấu hình mẫu, README, collection Postman, log và ảnh; ghi phản hồi khách hàng vào backlog.

7.4 Retrospective và cải tiến đề xuất

Nhóm cần tự ghi điều làm tốt, vướng mắc thực tế và hành động ở Sprint sau. Gợi ý review: thống nhất hash sau khi viết lại lịch sử; ghi tác giả code và người viết tài liệu riêng; đặt tên commit có task; lưu minh chứng kiểm thử kèm SHA; xác nhận test đồng thời trên cơ sở dữ liệu thật. Đây là đề xuất cho buổi họp, không phải biên bản đã diễn ra.

8 Các mục cần nhóm chốt sau review 1

1. Điền MSSV của Bảo Tín, Phú, Bảo Thiện, Lợi, Thuận và xác nhận ánh xạ tài khoản Git (đặc biệt Saturlen → Bảo Tín); không sửa tác giả lịch sử cho khớp bảng phân công.
2. Xác nhận GVHD, PO/SM, lịch Sprint, biên bản review và ngày đạt DoD từng task.
3. Bổ sung ảnh menu, chi tiết, giỏ, đăng nhập, lỗi thanh toán, biên lai, lịch sử và Runner vào báo cáo cuối.
4. Cập nhật log build/test tại một SHA thống nhất, đặc biệt test PostgreSQL đang có điều kiện chạy.

5. Chốt phần đặc tả Task 2/9 trên nhánh riêng có đưa vào bản bàn giao hay chỉ dẫn làm minh chứng đóng góp; không mặc định đã merge.

9 Nguồn và cách tái kiểm tra

- `docs/BrewLite-CongNghePhanMem-fixed.md`: đề, tiêu chí 10 task và DoD.
- `docs/Assigment_template_SGU/Assignment_report.tex`: mẫu trình bày SGU.
- `docs/minh-chung-10-task.md`: ánh xạ request/test/ảnh; có phần snapshot cũ nên bảng Git được tra lại.
- `docs/product-backlog.md`, `docs/scrum-process.md`: backlog và hồ sơ Scrum nháp.
- `docs/postman/results/2026-10-21-run.txt`: log Postman lịch sử.
- `git-evidence.txt`: dữ liệu Git trực tiếp phục vụ bản review này.

```
git show --format=fuller --stat <commit>
git branch -a --contains <commit>
git merge-base --is-ancestor <commit> HEAD
git show --no-patch <merge-commit>
```

Các lệnh kiểm tra nhánh chứa và quan hệ tổ tiên không xác định duy nhất nhánh tạo commit. Thông điệp merge là bằng chứng lịch sử nhánh nguồn; cần đọc cùng diff và bối cảnh viết lại hash.